AIMLZG557 - Artificial and Computational Intelligence

Situated Learning Assignment (EC-1) - Question 2, Individual Submission

B.1 Submission Form

Name: Neelam Devi	**ID No:** 2025AG05908
Industry / Domain: Consumer electronics - mobile handset software quality assurance	**Organisation Type:** Large multinational (electronics and mobile manufacturer)
Course: AIMLZG557, Semester S1-26	**Instructor In-charge:** Dr. V Indumathi
Instructor prompt: Identify a real workplace process where decisions are governed by predefined rules	**Process chosen:** Defect triage and release go / no-go decision

B.2 Anchor (1 mark)

Workplace process chosen: defect triage and release go / no-go decision for a smartphone software build.

I have spent about 14 years in mobile testing at Samsung. Before any new firmware or OTA build goes out to a carrier or to the field, the test team goes through every open defect and decides three things: how serious it is, who has to act on it, and finally whether the build as a whole is fit to be released. In practice the people in the review are the QA lead, the development lead and the release manager. The inputs are the defect tracker entries (severity, logs, reproduction details), the regression suite result and the carrier certification checklist.

This process suits rule-based reasoning because the decisions follow written criteria, for example a critical crash that can be reproduced stops the release, and a defect without logs goes back to the tester. The problem I have seen many times is that decisions are not consistent: the same defect can get a different treatment depending on who is reviewing it on a given day, and the reasoning behind a decision is often lost in email threads. A proper knowledge base would make the decisions repeatable and easy to explain.

Please note that the rules below are a simplified and generalised version of what is commonly followed in mobile QA. They are not the internal policy of any company, and the defect IDs and data are made up for this assignment.

B.3 Apply (2 marks)

B.3.1 Business rules in plain English

Rule	Statement
R1	A critical defect that can be reproduced and is still open is a release blocker.
R2	A critical defect that cannot be reproduced is not a blocker; it is kept under observation and more data is collected.
R3	Any open security defect is a release blocker.
R4	Any open defect that affects a carrier / certification test is a release blocker.
R5	An open high-severity defect that is a regression from the previous release gets priority P1.
R6	An open defect without logs attached is returned to the reporter.
R7	An open release blocker must be escalated to the development lead within 24 hours.
R8	If a fix has been delivered but QA has not verified it, the defect is pending verification.
R9	If a fix has been delivered and QA has verified it, the defect is closed.
R10	A release blocker that is not closed makes the build carry an open blocker.
R11	If a build carries an open blocker, the decision is No-Go.
R12	If a build has no open blocker and the regression pass rate is at least 95%, the decision is Go.

B.3.2 Propositional logic

Propositions used (each one is either true or false for a given defect or build). For this assignment I take "reproducible" to mean the failure was seen in at least 3 out of 5 attempts; the exact number differs from team to team.

Symbol	Meaning	Symbol	Meaning
C	Defect severity is Critical	Es	Escalate to dev lead within 24 h
H	Defect severity is High	Fx	Fix has been delivered
Rp	Defect is reproducible (3 of 5 attempts)	Vf	Fix verified by QA
Rg	Defect is a regression	Cl	Defect is closed
Sc	Defect is security related	Ob	Keep under observation
Ct	Defect affects carrier / certification test	Rt	Returned to reporter
Lg	Logs are attached	Pv	Pending verification
Op	Defect is open	P1	Priority P1 assigned
Bk	Defect is a release blocker	OB	Build carries an open blocker
PR	Regression pass rate is 95% or more	Go / NG	Go decision / No-Go decision

Rule	Propositional form
R1	C ∧ Rp ∧ Op → Bk
R2	C ∧ ¬Rp ∧ Op → Ob
R3	Sc ∧ Op → Bk
R4	Ct ∧ Op → Bk
R5	H ∧ Rg ∧ Op → P1
R6	Op ∧ ¬Lg → Rt
R7	Bk ∧ Op → Es
R8	Fx ∧ ¬Vf → Pv
R9	Fx ∧ Vf → Cl
R10	Bk ∧ ¬Cl → OB
R11	OB → NG
R12	¬OB ∧ PR → Go

The problem is that these symbols describe *one* defect. To use R1 for ten defects I would need symbols like C_D101, Rp_D101, C_D102 and so on, and a separate copy of every rule for every defect. Rules such as R10 to R12 talk about "some defect in the build" and cannot be written properly at all. So I wrote the same rules again using predicate logic below.

B.3.3 Predicate logic

Constants: Critical, High, Medium, Low (severity levels); B2 (a build); D101 to D107 (defects). **Variables:** d for a defect, b for a build, s for a severity level. **Predicates:**

Predicate	Meaning	Predicate	Meaning
Defect(d)	d is a defect	Open(d)	d is open
Build(b)	b is a build	Fixed(d), Verified(d)	fix delivered / verified
Sev(d, s)	d has severity s	Closed(d)	d is closed
Reproducible(d)	d reproducible	Blocker(d)	d is a release blocker
Regression(d)	d is a regression	Escalate(d)	escalate to dev lead

Predicate	Meaning	Predicate	Meaning
Security(d)	d is security related	Observe(d)	keep under observation
CertImpact(d)	affects certification test	Returned(d)	returned to reporter
HasLogs(d)	logs attached to d	PendingVerif(d)	waiting for QA verification
FoundIn(d, b)	d was found in build b	PriorityP1(d)	d has priority P1
BlocksBuild(d, b)	d is an unclosed blocker in b	PassRate95(b), Go(b), NoGo(b)	pass rate ≥ 95% / decisions

Rule	Predicate logic form
R1	∀d [Sev(d, Critical) ∧ Reproducible(d) ∧ Open(d) → Blocker(d)]
R2	∀d [Sev(d, Critical) ∧ ¬Reproducible(d) ∧ Open(d) → Observe(d)]
R3	∀d [Security(d) ∧ Open(d) → Blocker(d)]
R4	∀d [CertImpact(d) ∧ Open(d) → Blocker(d)]
R5	∀d [Sev(d, High) ∧ Regression(d) ∧ Open(d) → PriorityP1(d)]
R6	∀d [Open(d) ∧ ¬HasLogs(d) → Returned(d)]
R7	∀d [Blocker(d) ∧ Open(d) → Escalate(d)]
R8	∀d [Fixed(d) ∧ ¬Verified(d) → PendingVerif(d)]
R9	∀d [Fixed(d) ∧ Verified(d) → Closed(d)]
R10	∀d ∀b [Blocker(d) ∧ FoundIn(d, b) ∧ ¬Closed(d) → BlocksBuild(d, b)]
R11	∀b [Build(b) ∧ ∃d BlocksBuild(d, b) → NoGo(b)]
R12	∀b [Build(b) ∧ ¬∃d BlocksBuild(d, b) ∧ PassRate95(b) → Go(b)]

The universal quantifier (∀) lets one rule cover every defect and every build. The existential quantifier (∃) is what makes the build-level decision in R11 and R12 expressible: "there is at least one blocking defect in this build".

B.3.4 Knowledge base

The knowledge base has **37 facts** and **12 inference rules** (R1 to R12 above, in predicate form). Facts are given for build B2 and seven defects. Defect descriptions are only there to make the data readable; they are not part of the logic.

Defect	Description (made-up)	Facts in the KB	#
Build	Release candidate under review	Build(B2); ¬PassRate95(B2) (pass rate was 92%)	2
D101	Camera app crashes on launch after OTA update	Defect(D101); Sev(D101, Critical); Reproducible(D101); Open(D101); HasLogs(D101); FoundIn(D101, B2)	6
D102	Bluetooth pairing fails with some car kits (worked in previous release)	Defect(D102); Sev(D102, High); Regression(D102); Open(D102); HasLogs(D102); FoundIn(D102, B2)	6
D103	Random reboot seen once overnight, not repeated	Defect(D103); Sev(D103, Critical); ¬Reproducible(D103); Open(D103); HasLogs(D103); FoundIn(D103, B2)	6
D104	Lock screen can be bypassed from the notification shade	Defect(D104); Security(D104); Open(D104); HasLogs(D104); FoundIn(D104, B2)	5
D105	Higher battery drain reported by a tester, no logs	Defect(D105); Sev(D105, Medium); Open(D105); ¬HasLogs(D105)	4
D106	Wi-Fi drops after long idle, developer fix delivered	Defect(D106); Sev(D106, High); Fixed(D106); ¬Verified(D106)	4

Defect	Description (made-up)	Facts in the KB	#
D107	Text clipped in a settings screen, fixed and checked	Defect(D107); Sev(D107, Low); Fixed(D107); Verified(D107)	4

Total: 2 + 6 + 6 + 6 + 5 + 4 + 4 + 4 = 37 facts.

Assumption: I use the closed-world assumption, meaning a fact that is not in the KB (and cannot be derived) is treated as false. For example there is no Closed(D101) fact, so D101 is not closed. Some negative facts such as ¬Reproducible(D103) are still written out explicitly so that the data is easy to read; under the closed-world assumption they would also follow from the fact being absent. Rules R2, R6, R8 and R12 use negation, which is evaluated this way (negation as failure).

B.3.5 Reasoning: five workplace queries

Query 1: Is D101 a release blocker, and does it need escalation?

- Sev(D101, Critical), Reproducible(D101) and Open(D101) are all facts, so R1 gives **Blocker(D101)**.
- Blocker(D101) and Open(D101) together trigger R7, giving **Escalate(D101)**.
- *Method:* universal instantiation (d = D101 in R1 and R7), then modus ponens, applied twice in sequence (forward chaining).
- **Answer:** yes, D101 is a blocker and must go to the dev lead within 24 hours.

Query 2: Is D103 (the one-time reboot) a release blocker?

- R1 needs Reproducible(D103), but the KB has ¬Reproducible(D103), so R1 does not fire. R3 and R4 also do not fire because D103 has no Security or CertImpact fact.
- R2 fires: Sev(D103, Critical), ¬Reproducible(D103), Open(D103) give **Observe(D103)**.
- *Method:* the query is answered by checking which rules can fire; the one rule whose conditions are fully satisfied is R2. A rule with a false condition (R1) gives no conclusion.
- **Answer:** no, it is not a blocker now. It stays under observation and becomes a blocker (via R1) as soon as Reproducible(D103) is added.

Query 3: What happens to the lock-screen bypass, D104?

- Security(D104) and Open(D104) give **Blocker(D104)** by R3. This holds even though no severity fact is recorded for it.
- R7 then gives **Escalate(D104)**.
- *Method:* forward chaining with modus ponens. This case also shows why predicates help: R3 does not care about severity at all, so a defect can be a blocker through a different route than R1.
- **Answer:** D104 is a blocker and is escalated, purely because it is a security issue.

Query 4: Can build B2 be released?

- R10 with Blocker(D101), FoundIn(D101, B2) and no Closed(D101) gives BlocksBuild(D101, B2). In the same way R10 gives BlocksBuild(D104, B2).
- Because such a d exists, R11 gives **NoGo(B2)**. R12 cannot fire, since it needs no blocking defect and PassRate95(B2), and neither holds.
- D102 (P1 by R5) and D103 (observe) are not blockers, so they do not add to this result.
- *Method:* this is the goal-driven version. Asking "Go(B2)?" (backward chaining) leads to R12, which needs no blocking defect and PassRate95(B2); both sub-goals fail. Forward chaining from the facts reaches NoGo(B2) through R10 and R11, using the existential quantifier in R11.
- **Answer:** No-Go. What-if: if D101 and D104 were fixed and verified (R9 closes them) and the pass rate rose to 95%, R12 would give Go(B2).

Query 5: What is the status of D105, D106 and D107?

- D105: Open(D105) and ¬HasLogs(D105) give **Returned(D105)** by R6. The tester has to attach logs first.
- D106: Fixed(D106) and ¬Verified(D106) give **PendingVerif(D106)** by R8. It stays on the QA list.
- D107: Fixed(D107) and Verified(D107) give **Closed(D107)** by R9.
- **Answer:** D105 goes back to the reporter, D106 waits for verification, D107 is closed.

B.4 Analyse (1 mark)

B.4.1 Predicate logic compared with propositional logic

- **Generality.** With propositional logic I needed a separate set of symbols for every defect. With 7 defects and 12 rules that is already close to 60 rule instances; a real build has hundreds of defects. In predicate logic each rule is written once with the variable d and applies to any defect, including ones added later.
- **Relations between objects.** FoundIn(d, b) links a defect to a build. Propositional logic cannot say which build a defect belongs to, so it could not support the release decision at all.
- **Quantifiers.** The Go / No-Go rule is really "no open blocker exists in this build". That is an ∃ statement. In propositional logic I had to invent a single symbol OB and set it by hand.
- **Reasoning power.** Queries 1 to 5 were answered by plain substitution of constants into the ∀ rules. The same KB can also answer "which defects in B2 are blockers?" by finding all d that satisfy Blocker(d), which a propositional version cannot do without listing every defect by hand.

B.4.2 Checking the KB against what the workplace needs

To test the KB, I first wrote down what a QA lead would decide by hand for each item in the sample data, and then compared it with what the KB derived.

Item	Decision by hand	KB output	Match
D101	Blocker, escalate	Blocker, Escalate (R1, R7)	Yes
D102	P1, but not a blocker	PriorityP1 (R5)	Yes
D103	Watch, collect more data	Observe (R2)	Yes
D104	Blocker, escalate	Blocker, Escalate (R3, R7)	Yes
D105	Send back for logs	Returned (R6)	Yes
D106	Waiting for QA check	PendingVerif (R8)	Yes
D107	Close	Closed (R9)	Yes
Build B2	No-Go	NoGo (R10, R11)	Yes

This is only a sanity check on seven made-up defects, not proof that the rules are right for a real project. A real validation would replay the KB on the defect lists of past releases and compare it with the decisions that were actually taken. Next, I compare what the review meeting really needs with what the KB provides:

What the workplace needs	What the KB gives	Gap
Same decision for the same defect, whoever reviews it	Rules are applied identically every time	Only as good as the input facts
Reasons that can be explained to dev and management	Each conclusion traces to a rule and facts	No plain-language explanation is generated yet
Fast triage when hundreds of defects are open	Derivation is instant once facts exist	Facts are entered by hand here
Room for judgment, e.g. a known blocker with a workaround	Not supported	Needs a waiver mechanism
Prioritisation that affects the release	P1 is derived (R5)	P1 does not influence Go / No-Go; a missing rule

B.4.3 Effectiveness of the knowledge base and what is missing

The KB handles the clear cases well. For all five queries it gave the same answer a QA lead would give, and every conclusion can be traced back to specific rules and facts, which is something email-based decisions do not offer. It also exposes gaps: D104 could be treated as a blocker without a severity value, and D105 was caught for missing logs before anyone spent time on it. But it is not complete enough for real use:

- **Severity and reproducibility are judgments.** "Reproducible" is really a ratio (for example 2 of 5 attempts), and severity is decided by a person. The KB treats them as fixed true/false facts, so a wrong input gives a wrong decision.
- **No exceptions or waivers.** In real projects a release manager sometimes accepts a known blocker with a workaround or a planned patch. The rules are strict and cannot represent this.
- **No impact data.** Number of affected users, device models, region or carrier are not modelled. A crash on one rarely used model and a crash on the flagship model are treated the same.
- **No time reasoning.** The 24-hour escalation in R7 is only stated as a conclusion; the KB cannot check whether the deadline was actually met.
- **Closed-world assumption.** If someone forgets to enter HasLogs or Security for a defect, the KB silently treats it as false and may miss a blocker.
- **Moving defects.** Facts change daily as defects are fixed or reopened, so the KB is only valid for the moment it was last updated.

B.5 Reflect (1 mark)

B.5.1 Practical constraints in my organisation

- **Incomplete and inconsistent data.** Defect entries are written by many testers, and fields like severity, logs and reproduction details are often filled in late or differently. Rules are only as good as the facts they run on.
- **Rules keep changing.** Carrier requirements and internal release criteria change from one project and one release to the next. Someone must own the rule set and update it, otherwise the KB becomes outdated and people stop trusting it.
- **Privacy and confidentiality.** Defect logs can contain user data and unreleased product information. A reasoning system reading them would need access control and should not copy data outside the secure tracker.
- **Scale and integration.** A real tracker holds tens of thousands of defects across many models and builds. The KB would have to read from the tracker automatically; manual entry of facts as I did here would not survive a real project.
- **People and process.** Leads may not accept an automatic No-Go for a build they want to ship, and the decision involves business judgment which logic alone does not capture.
- **Technology limits.** Full first-order logic is not decidable in general, so a real system would keep the rules to simple if-then form like mine and use a rule engine or Prolog-style tool. That keeps reasoning fast, but it limits what can be expressed (no uncertainty, no probabilities).

B.5.2 The single most important improvement

My recommendation: add a controlled waiver (exception) mechanism with human approval and an audit trail, so that the KB gives a recommendation and a person makes the final call.

I chose this over better data quality or more rules because of how release decisions are actually made. In my experience a build is sometimes shipped with a known blocker when there is a workaround, the affected population is small, or a patch is already planned. If the KB can only say No-Go, people will either ignore it or work around it, and then it is worse than useless because it gives a false sense of control. With a waiver step, the rule output stays as the default (R1 to R12 still say Blocker and No-Go), but a named approver can record an exception with a reason, which the KB stores as a new fact such as Waived(D101, B2, approver, reason). The Go / No-Go rule would then ignore waived blockers while the report still lists them. This keeps

the consistency benefit of the rules, keeps the final accountability with people, and leaves a record that can be reviewed after release if the field data shows a problem.